

PRACTICAL-1

```
#include <iostream> // used for input and output

#include <vector> //used to create dynamic arrays

#include <algorithm> //provides built in functions like swap()

#include <cstdlib> //provides rand() srand() functions

#include <ctime> //used with tim(0) for generating the random numbers each run

#include <chrono> // measures execution time


using namespace std;

using namespace chrono;


// Bubble Sort code

void bubbleSort(vector<int> &arr) // sorting function
{
    for (int i = 0; i < arr.size() - 1; i++) //outer loop.... controls the number of passes
    {
        for (int j = 0; j < arr.size() - i - 1; j++) //inner loop for comparing adjacent elements
        {
            if (arr[j] > arr[j + 1]) // if left element is greater than right then swap them
                swap(arr[j], arr[j + 1]);
        }
    }
}


// Selection Sort

void selectionSort(vector<int> &arr) // funtion for finding smallest element and places
it at the correct position
{
    for (int i = 0; i < arr.size() - 1; i++)
```

PRACTICAL-1

```
{
    int minIndex = i; // assume first element as minimum

    for (int j = i + 1; j < arr.size(); j++)
    {
        if (arr[j] < arr[minIndex]) //swap another element is greater than that
            minIndex = j;
    }

    swap(arr[i], arr[minIndex]);
}

// Insertion Sort
void insertionSort(vector<int> &arr) // sorting function first element is already sorted
{
    for (int i = 1; i < arr.size(); i++)
    {
        int key = arr[i]; // check with the adjacent elements
        int j = i - 1;

        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j]; // move larger elements to the right
            j--;
        }

        arr[j + 1] = key; // inserting the elements
    }
}
```

PRACTICAL-1

```
    }  
}  
  
// Merge Sort  
void merge(vector<int> &arr, int left, int mid, int right)  
{  
    vector<int> temp; // dividing the array  
  
    int i = left;  
    int j = mid + 1;  
  
    while (i <= mid && j <= right)  
    {  
        if (arr[i] < arr[j])  
            temp.push_back(arr[i++]); //compares both values and smaller value goes into  
temp  
        else  
            temp.push_back(arr[j++]);  
    }  
  
    while (i <= mid)  
        temp.push_back(arr[i++]);  
  
    while (j <= right)  
        temp.push_back(arr[j++]);  
  
    for (int k = left; k <= right; k++)  
        arr[k] = temp[k - left];  
}
```

PRACTICAL-1

```
void mergeSort(vector<int> &arr, int left, int right)
```

```
{  
    if (left < right)  
    {  
        int mid = (left + right) / 2;  
  
        mergeSort(arr, left, mid);  
        mergeSort(arr, mid + 1, right);  
  
        merge(arr, left, mid, right);  
    }  
}
```

```
// Quick Sort
```

```
int partition(vector<int> &arr, int low, int high)  
{  
    int pivot = arr[high]; //selecting pivot element  
    int i = low - 1;  
  
    for (int j = low; j < high; j++)  
    {  
        if (arr[j] <= pivot) //compare elements with the pivot elements  
        {  
            i++;  
            swap(arr[i], arr[j]);  
        }  
    }  
}
```

PRACTICAL-1

```
    swap(arr[i + 1], arr[high]);  
    return i + 1;  
}
```

```
void quickSort(vector<int> &arr, int low, int high)  
{  
    if (low < high)  
    {  
        int p = partition(arr, low, high);  
  
        quickSort(arr, low, p - 1);  
        quickSort(arr, p + 1, high);  
    }  
}
```

```
// Function to calculate execution time  
void calculateTime(string name, vector<int> arr, void (*sortFunc)(vector<int>&))  
{  
    auto start = high_resolution_clock::now();  
  
    sortFunc(arr);  
  
    auto stop = high_resolution_clock::now();  
  
    cout << name << " : "  
        << duration_cast<microseconds>(stop - start).count()  
        << " microseconds" << endl;
```

PRACTICAL-1

```
}

int main()
{
    srand(time(0)); //for generating random numbers every run

    int n = 100;

    vector<int> arr(n);

    for (int i = 0; i < n; i++)
        arr[i] = rand() % 1000; generates random numbers from 0 to 999

    cout << "Number of Elements = " << n << endl << endl;

    calculateTime("Bubble Sort", arr, bubbleSort);
    calculateTime("Selection Sort", arr, selectionSort);
    calculateTime("Insertion Sort", arr, insertionSort);

    vector<int> temp;

    temp = arr;
    auto start = high_resolution_clock::now();
    mergeSort(temp, 0, n - 1);
    auto stop = high_resolution_clock::now();
    cout << "Merge Sort : "
        << duration_cast<microseconds>(stop - start).count()
        << " microseconds" << endl;
```

PRACTICAL-1

```
temp = arr;
start = high_resolution_clock::now();
quickSort(temp, 0, n - 1);
stop = high_resolution_clock::now();
cout << "Quick Sort : "
      << duration_cast<microseconds>(stop - start).count()
      << " microseconds" << endl;

return 0;
}
```